

# Types — A Readable Companion to the Elixir Docs

Chapter 12 · checked against Elixir 1.20 · Sep 26, 2026

## 1. Why types in Elixir are in transition

Elixir is dynamically typed: values carry their types at run time, and pattern matching and guards check them where it matters. For years, the only static checking came from *typespecs* plus Dialyzer, an Erlang tool that reports provable contradictions and nothing else (Erlang book, chapter 8).

That is changing. Elixir is gaining a *gradual set-theoretic type system* built into the compiler. The work began landing in the 1.17 era and has grown with each release; version 1.20 infers types from guards, from how function bodies use their arguments, and across `case`, `cond` and `with` clauses. The docs describe it as sound, gradual and still a milestone in progress: today it only *infers*; user-written type signatures are planned but “absent.”

So a codebase today has two separate type notations with different jobs, and it is easy to believe one of them is doing something it isn’t.

## 2. The building blocks

### What checks what

| Mechanism                                              | Written by you? | Checked by                                      | Catches                                                 | Misses                                                                     |
|--------------------------------------------------------|-----------------|-------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| Pattern matching and guards                            | Yes             | The runtime, every call                         | Wrong shapes, at the point of entry                     | Nothing that passes the pattern                                            |
| Typespecs ( <code>@spec</code> , <code>@type</code> )  | Yes             | Nothing in the compiler; Dialyzer if you run it | Contradictions Dialyzer can prove                       | Anything merely <i>possible</i> ; bugs hidden by specs that are too narrow |
| Set-theoretic inference (1.17+, much expanded in 1.20) | No: inferred    | The compiler, on every build                    | Operations that can never succeed on the inferred types | Anything typed <code>dynamic()</code> ; it is best-effort                  |
| <code>@enforce_keys</code> , <code>struct!/2</code>    | Yes             | Runtime, at construction                        | Missing keys                                            | Wrong value types (chapter 1)                                              |

### Typespecs in brief

```
@type status :: :active | :suspended
@type t :: %__MODULE__ {id: pos_integer(), status: status()}
```

```
@spec suspend(t()) :: {:ok, t()} | {:error, :already_suspended}
```

The docs are clear about their role: typespecs are for documentation and for tools like Dialyzer, and “may be phased out as the set-theoretic type effort moves forward.” The compiler does not use them to check or optimise code.

## 3. How it actually works

### Specs are claims, not checks

A typespec is never enforced at run time. `@type t :: %User{age: integer()}` does not stop `%User{age: "42"}` from existing. Dialyzer only reports cases where your code *cannot* satisfy the spec, and it trusts specs that overlap with what it infers, so a spec that is too narrow can silence real problems (Erlang book, chapter 8, section 4.5).

### Inference is gradual

The new type system gives values it can't pin down the type `dynamic()`, and doesn't complain about them. Within a project, calls between your own modules are treated as `dynamic()` for now (to avoid cascading recompilation), and the docs note it “doesn't guarantee it will find all possible type incompatibilities” and can produce false positives in some constructs. A clean compile is evidence, not proof.

### Runtime checks are still the contract

Because neither system is complete, the guarantee at a module's boundary is still what its patterns and guards enforce. Data from outside (JSON, forms, databases) must be validated where it enters.

## 4. Plausible but wrong

### 4.1 Treating a spec as validation

```
@spec create(%{name: String.t(), age: non_neg_integer()}) :: {:ok, User.t()}  
def create(params), do: {:ok, struct!(User, params)}
```

The spec says `age` is a non-negative integer, but nothing checks it. `%{"age" => "-5"}` from a form flows straight into the struct. Fix: validate at the boundary (a changeset, or explicit guards and conversions) and return `{:error, reason}` for bad input. Keep the spec as documentation of the validated result.

### 4.2 A spec narrower than the code

```
@spec fetch(id()) :: User.t()  
def fetch(id) do  
  case Repo.get(User, id) do
```

```

nil -> {:error, :not_found}
user -> user
end
end

```

The spec omits `{:error, :not_found}`. Dialyzer accepts it because the types overlap, then analyses callers as if errors can't happen, so a caller that correctly handles the error can get a “pattern can never match” warning, inviting someone to delete it. Fix: write the real return type, `User.t() | {:error, :not_found}`, and treat narrowing specs to silence warnings as a bug.

### 4.3 Specs made of `term()`

```
@spec process(term(), keyword()) :: term()
```

This passes every check and documents nothing. Generated code often looks like this because it is always “correct.” Fix: specify what the function actually takes and returns, even approximately (`map()`, `{:ok, Order.t()} | {:error, atom()}`), or leave the spec out rather than pretend.

### 4.4 Trusting a clean compile

```
def label(%{status: status}), do: "Status: " <> status # status is an atom
```

Whether the compiler warns here depends on what it can infer about the map at the call sites. If the value comes from another module in your project, or from decoded JSON, it is `dynamic()` and there is no warning. The bug appears at run time as `ArgumentError`. Fix: tests for the unhappy paths (chapter 11) and explicit conversions (`Atom.to_string/1` or interpolation), not reliance on the checker.

### 4.5 A growing Dialyzer ignore file

```
.dialyzer_ignore.exs: 140 entries
```

Every entry was added to make CI green “for now.” Dialyzer warnings are almost never false alarms, because it only reports what it can prove. The ignore file becomes a list of unexamined bugs. Fix: fix or justify each entry with a comment, and fail CI on new warnings.

### 4.6 Opaque types used as open maps

```

@opaque t :: %{items: list(), total: integer()}
...
# in another module
cart.total

```

An `@opaque` type promises that other modules won't look inside. Reaching in works at run time, and Dialyzer reports it. It also ties the other module to the internal shape, which `@opaque` exists to prevent. Fix: expose functions (`Cart.total(cart)`), or make the type public with `@type` if the structure really is the API.

## 5. Review checklist and sources

---

- ❑ External data is validated at the boundary; specs are not relied on as checks
- ❑ Specs describe the real inputs and outputs, including every error shape
- ❑ No blanket `term()` specs on public functions
- ❑ Compiler type warnings are treated as errors in CI; a clean compile is not treated as proof
- ❑ Dialyzer runs in CI; ignore entries are few and justified
- ❑ `@opaque` types are only accessed through their module's functions

### Sources

- Gradual set-theoretic types: <https://hexdocs.pm/elixir/gradual-set-theoretic-types.html>
- Typespecs reference: <https://hexdocs.pm/elixir/typespecs.html>
- Elixir changelog: <https://hexdocs.pm/elixir/changelog.html>
- Dialyzer User's Guide: [https://www.erlang.org/doc/apps/dialyzer/dialyzer\\_chapter.html](https://www.erlang.org/doc/apps/dialyzer/dialyzer_chapter.html)
- Erlang book, chapter 8 (Testing, on Dialyzer): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>